

Quantum implementation of Keccak- $f(25)$ for emulator and quantum hardware, and application to quantum password cracking

Mathilde Chenu^{*1} and Mario Chizzini¹

¹Joint Research Centre

Abstract

In this work, we present a quantum implementation of Keccak- $f(25)$, a toy version of SHA-3 introduced in the specification of Keccak [BDPA11], with the objective of using as few qubits as possible so that the resulting implementation can be run both on quantum emulators and on quantum hardware. The code, written in Qiskit, uses 25 qubits representing the internal state. All computations are performed in-place, meaning that no ancillary qubit is required. We use this implementation to compare the number of logical and instruction-set-architecture quantum gates required to run Keccak- $f(25)$, both on emulators and on quantum hardware, as well as for Grover’s algorithm on the hash function. Finally, we use our implementation to highlight the specificity of quantum password cracking using Grover’s algorithm with Keccak- $f(25)$.

1 Introduction

1.1 Context

Gate-based quantum computers are still imperfect, due to limited qubits and noisy operations, but the technology is rapidly evolving. As quantum attacks have the potential to impact traditional cryptographic algorithms, they should be studied and understood well in advance, both to monitor their evolving feasibility and to plan adequate counter-measures.

Cryptographic hash functions are mostly vulnerable to Grover’s algorithm attacks, which reduces the exhaustive search attack over a domain of size N from $\mathcal{O}(N)$ classical operations to $\mathcal{O}(\sqrt{N})$ quantum operations [Gro96]. While these attacks remain unfeasible for now due to the large number of qubits needed and the extensive number of operations, monitoring the advances is crucial to be prepared for when it becomes feasible (see [DC24] for an overview, but also [AMG⁺16][GLRS16][JNRV20][GM25]).

Cryptographic hash functions often operate on a large internal state whose length varies from 128 bits to 1600 bits. For example, the internal state has 1600 bits for SHA-3 [BDPA11], 256 bits for SHA-256 [NIS] and Argon2 [BDK15], 160 bits for SHA-1 [NIS] and 128 bits for MD5 [Riv92]. To represent this internal state, the quantum implementation of a hash function requires at least as many qubits as the number of bits in the internal state. This limits the possibility of implementing them on currently available gate-based quantum hardware, which typically have up to a hundred qubits: IBM Nighthawk quantum computer has 120 superconducting-circuit-based qubits, Google’s Willow has 105 superconducting transmon qubits, and Quantinuum’s Helios has 98 ion-trapped-based qubits. This limitation is even more salient for quantum emulators¹, for which the required memory doubles with each qubit emulated: a CPU (Intel Core i7 CPU with 16GB) can emulate up to 30 qubits, a node of 8 GPUs (NVIDIA A100 with 320GB) can reach 35 qubits [Che25], and 2048 GPUs (NVIDIA A100 with 320GB) top up at 40 qubits [WWJ⁺23].

^{*}mathilde.chenu@ec.europa.eu

¹Quantum emulators, which reproduce the behaviour of gate-based quantum computers classically, are often called “simulators” in the computer science community. However, quantum physicists use the term “quantum simulator” to refer to hardware capable of modelling the behaviour of quantum particles. To avoid confusion, we will use the term “emulator” consistently throughout this work.

1.2 Contributions

Given the constraints on the number of qubits available on both emulators and quantum hardware, we chose to study the toy version of SHA-3, Keccak-f(25), introduced in the Keccak reference [BDPA11]. With its internal state of 25 qubits, Keccak-f(25) can be implemented even on a PC-based emulator, allowing to study the hash algorithm and its associated Grover’s algorithm. The implementation also runs on quantum hardware, allowing us to study how the logical qubits and operations are mapped onto the physical qubits and operation set available on the hardware, as well as the noise propagation. We chose to use the Qiskit language as it allows a versatility of execution: the code is the same whether the circuit is run on an emulator or on quantum hardware. For our experiments on quantum machines, we used the IBM Quantum Platform under their free plan, using the machines ibm-torino, ibm-marrakesh and ibm-fez, all of which rely on Heron quantum processors. We use two different metrics to report gate counts: the number of logical gates (CCX, CX, X, H, P), corresponding to the mathematical operations double-controlled NOT, single-controlled NOT (or XOR), NOT, the Hadamard gate H and the phase gate P respectively ; and the number of ISA gates corresponding to the Instruction Set Architecture (ISA) of the quantum machines used (CZ, RZ, SX and X) for the controlled rotation around the Z axis, the single-qubit rotation around the Z axis, the square root gate and the NOT gate respectively.

We present our results as follows: we detail our 25-qubits implementation of Keccak-f(25) in Section 2. In Section 3, we detail the number of logical gates (CCNOT, CNOT, NOT, H, P) and ISA gates (CZ, RZ, SX and X) used on the emulator to run the different functions of Keccak-f(25), as well as the number of ISA gates, the timings and the noise propagation on the quantum hardware. In Section 4, we provide both logical gate and ISA gate counts for the execution of Grover’s algorithm with Keccak, on emulators and quantum hardware. Finally, in Section 5, we use our implementation of Keccak-f(25) to study the specificity of quantum password cracking with Grover’s algorithm.

The code is available at <https://github.com/MathildeChenu/grover-keccakf25.git>.

Our work allows to step between the logical layer and the physical layer of the specificity of running Keccak and Grover on a quantum hardware, thanks to the possibility to alternate between perfectly emulated logical qubits and real, although imperfect, quantum hardware. While using only a toy example of Keccak, our work provides a better understanding of SHA-3 quantum implementations, paving the way for possible implementation optimisations, and a better understanding of how theoretical algorithms are mapped onto quantum hardware.

1.3 Related works

The quantum circuits of several cryptographic hash functions have been studied in the literature (see Table 1), for example SHA-3 ([AMG⁺16], [Pre22], [SJS23], [LKL⁺24], [SP25], [GE25]), SHA-2 ([KHJ18] [Pre22]), SHA-1 ([Pre22] [DBK24] [JLO⁺25]), MD5 ([Pre22][DBK24][CSYS25]), Blake and Chacha [RBL⁺21], and various lightweight hash functions [LJS⁺22].

Focusing on the works on SHA-3, these circuits are used to establish logical-gate counts, either with purely theoretical constructs such as in [AMG⁺16], [SJS23], [LKL⁺24], [SP25], or using emulators such as Q#, Qibo, ProjectQ, Qiskit in [Pre22], [GE25].

Most commonly, these works follow two different approaches: either the minimisation of the depth of the circuit ([SJS23], [GE25]), or the minimisation of the width of the circuit, that is the number of qubits used ([AMG⁺16], [Pre22], [LKL⁺24], [SP25]). Given our constraints on the number of qubits available on emulators and quantum hardware, we focus on this qubit minimisation approach. While the work of [LKL⁺24] shows that Keccak-f(n), and in particular SHA-3, which is an instance of Keccak-f(1600), can be implemented using only the n qubits from the internal register and no additional ancillaries, they do not extend their gate-count analysis to Grover’s algorithm. The work from [SP25] does take into consideration Grover’s algorithm on Keccak, but uses more than n qubits. The gate-count analysis of these two papers is theoretical and does not provide implementations.

We bridge this gap by providing an in-place², 25-qubits implementation of Keccak-f(25) and of Grover’s algorithm on Keccak-f(25). We run our implementation both on the AerSimulator quantum emulator to establish the logical-gate count, and on IBM quantum machines to establish a ISA-gate count. To the best of our knowledge, this is the first instance of Grover’s algorithm on the Keccak hash function being implemented with a qubit minimisation approach, and run both on an emulator and quantum hardware.

²We use “in-place” in the sense that the implementation does not use more qubits than the ones required to represent the internal state.

Paper	Algorithms	Platform	Minimisation	Gate-count metric	Includes Grover
[AMG ⁺ 16]	SHA-3, SHA-2	Theoretical	None	Logical	Yes
[KHJ18]	SHA-2	Theoretical	Balancing	Logical	Yes
[RBL ⁺ 21]	Blake, ChaCha	Emulator (Qibo)	Qubits	Logical	Yes
[Pre22]	SHA-1, MD5	Emulator (Q#)	Qubits	Logical	Yes
[LJS ⁺ 22]	Lightweight hashes	Emulator (ProjectQ)	Qubits	Logical	No
[SJS23]	SHA-3	Theoretical	Depth	Logical	No
[DBK24]	SHA-1, MD5	Emulator (Qiskit)	Qubits	Logical	No
[LKL ⁺ 24]	SHA-3	Theoretical	Balancing	Logical	No
[SP25]	SHA-3	Theoretical	Qubits	Logical	Yes
[GE25]	SHA-3	Emulator (Qiskit)	Depth	Logical	Yes
[CSYS25]	MD5	Emulator (ProjectQ)	Depth	Logical	No
[YSJ ⁺ 25]	SHA-1	Emulator (ProjectQ)	Depth	Logical	No
[JLO ⁺ 25]	SHA-3, SHA-2	Theoretical	Depth	Logical	Yes
This work	Keccak f(25)	Emulator (Qiskit) Quantum hardware	Qubits	Logical ISA	Yes

Table 1: Comparison between previous works on quantum implementations of hash functions and this work

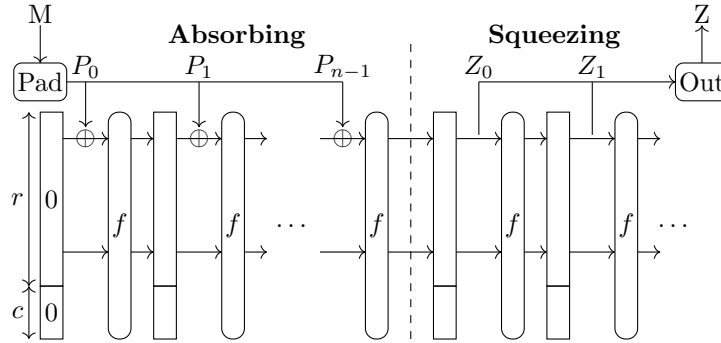


Figure 1: Sponge construction with rate r and capacity c operating on a state of $b = r + c$ bits.

2 Keccak-f(25) implementation choices and details

Keccak [BDPA11] is the winning candidate of the hashing algorithm SHA-3 competition organised by the NIST between 2007 and 2012 [CPB⁺]. It has a sponge structure, with an internal state on which is applied a pseudorandom permutation function to absorb an incoming bitstream, then squeeze out a hash (see Figure 1). We refer to the Keccak documentation for more details on the sponge construction [BDPA11].

For Keccak, the permutation function consists of 12 to 24 rounds of five operations θ , π , ι , χ and ρ applied to the internal state. The size of the internal state varies depending on the version of Keccak: Keccak-f(n) operates on a n -bit internal state. For example, SHA-3 is an instance of Keccak-f(1600) operating on a 1600-bit state. The internal state is generally represented as a $5 \times 5 \times \frac{n}{25}$ matrix.

Keccak-f(25) was first introduced in the Keccak submission [BDPA11], as the smallest Keccak instance useful to study the properties of the pseudorandom permutation. It consists of an internal state of $5 \times 5 \times 1 = 25$ -bit on which the five building block functions of Keccak θ , π , ι , χ and ρ operate 12 rounds. In the context of quantum computing, its small internal state allows its implementation on both quantum emulators and quantum hardware, while giving an insight into the quantum implementation of larger Keccak functions, in particular SHA-3.

Our quantum implementation of Keccak-f(25) aims at reducing as much as possible the number of qubits to ensure that the code can be executed on an emulator and on quantum hardware. For the non-linear function χ , we use the in-place implementation of [LKL⁺24]. The work from [PMH08] showed that any linear reversible transformations on an input of size n could theoretically be implemented in-place on quantum devices using only n qubits, meaning that for Keccak, the linear functions θ , π , ι and ρ could be implemented in-place on the internal state without using ancillary qubits. While [AMG⁺16] were the first to provide in-place implementations of π , ι and ρ , to the best of our knowledge, an in-place implementation of θ had not been published before this work.

Thanks to the in-place θ circuit, we obtain an in-place 25-qubits implementation of Keccak f(25).

We describe below how the five operations θ , π , ι , χ , and ρ have been implemented with the minimisation of the number of qubits used in mind. Since the Keccak function will be used within Grover's algorithm, we also describe how the reverse function is implemented for each step. In the case of Keccak-f(25), the internal state s consists of 25 qubits which are represented in a 5×5 matrix as in Figure 2.

		$x = 0$	$x = 1$	$x = 2$	$x = 3$	$x = 4$
		C_0	C_1	C_2	C_3	C_4
$y = 0$	L_0	q_0	q_1	q_2	q_3	q_4
$y = 1$	L_1	q_5	q_6	q_7	q_8	q_9
$y = 2$	L_2	q_{10}	q_{11}	q_{12}	q_{13}	q_{14}
$y = 3$	L_3	q_{15}	q_{16}	q_{17}	q_{18}	q_{19}
$y = 4$	L_4	q_{20}	q_{21}	q_{22}	q_{23}	q_{24}

Figure 2: Internal state s with qubits correspondence. The qubit at indices $[i, j]$ in the table representation corresponds to the qubit at index $[5i + j]$ in the list $\{q_0, \dots, q_{25}\}$

2.1 θ function

The θ function adds to each bit of the internal state s in its matrix representation the parity of the columns on its left and on its right, following the matrix representation of Figure 2. Because the parity of the columns is changed during the computation, we write $\text{parity}(C_j^{init})$, the parity of the column j at the beginning of the θ operation, and $\text{parity}(C_j)$, the current parity. The θ operation can then be written, for i and j in $\mathbb{Z}/5\mathbb{Z}$, as:

$$s[i][j] \leftarrow s[i][j] \oplus \text{parity}(C_{j-1}^{init}) \oplus \text{parity}(C_{j+1}^{init}) .$$

When adding the parity of a column, say C_a , to the qubits of another, say C_b , the constraint of not using more qubits than the 25 qubits implies that the value of C_a^{init} needs to be expressed in terms of the current values of the columns of the internal state, except C_b . Algorithm 1 provides a high-level view of our in-place implementation strategy for θ , while the procedure to add the parity of a column is detailed in Algorithm 2.

More precisely, in Algorithm 1, we have

- $C_4 = C_4^{init} \oplus \text{parity}(C_3^{init})$ after line 1,
- $C_3 = C_3^{init} \oplus \text{parity}(C_2^{init})$ after line 2,
- $C_2 = C_2^{init} \oplus \text{parity}(C_1^{init})$ after line 3,
- $C_1 = C_1^{init} \oplus \text{parity}(C_0^{init})$ after line 4,
- and finally $C_4 = C_4^{init} \oplus \text{parity}(C_3^{init}) \oplus \text{parity}(C_0^{init})$ after line 5.

The linear combinations of the current values C_0, C_1, C_2, C_3 and C_4 of the columns are then used to retrieve the initial values $C_0^{init}, C_1^{init}, C_2^{init}$ and C_3^{init} , and compute the remaining operations for θ .

It appears from Algorithms 1 and 2 that the computation of the function θ only requires CNOT gates. The reverse function is computed by performing the same succession of `add_parity` steps in reverse order.

2.2 π function

The π function is a permutation on the qubits of the internal state, such that

$$s[i][j] \leftarrow s[j][(2i + 3j) \bmod 5] .$$

The permutation has two orbits: one containing $s[0][0]$, and the other containing the rest of the permutations on the internal state. This operation can be performed using the SWAP gate, whose

Algorithm 1: θ algorithm for quantum hardware

Data: internal state s

```
1 add_parity(s, 3, 4) ;
2 add_parity(s, 2, 3) ;
3 add_parity(s, 1, 2) ;
4 add_parity(s, 0, 1) ;
5 add_parity(s, 0, 4) ;
6 # Here  $C_4 = C_4^{init} \oplus \text{parity}(C_3^{init}) \oplus \text{parity}(C_0^{init})$ ,  $C_0 = C_0^{init}$ , and for
    $j \in \{1, 2, 3\}$ ,  $C_i = C_i^{init} \oplus \text{parity}(C_{i-1}^{init})$ ;
7 #  $\text{parity}(C_1) \oplus \text{parity}(C_2) \oplus \text{parity}(C_3) \oplus \text{parity}(C_4) = \text{parity}(C_4^{init})$  added to  $C_0$ ;
8 add_parity(s, 1, 0) ;
9 add_parity(s, 2, 0) ;
10 add_parity(s, 3, 0) ;
11 add_parity(s, 4, 0) ;
12 #  $\text{parity}(C_0) \oplus \text{parity}(C_4) = \text{parity}(C_3^{init})$  added to  $C_2$ ;
13 add_parity(s, 0, 2) ;
14 add_parity(s, 4, 2) ;
15 #  $\text{parity}(C_2) \oplus \text{parity}(C_3) = \text{parity}(C_1^{init})$  added to  $C_0$ ;
16 add_parity(s, 2, 0) ;
17 add_parity(s, 3, 0) ;
18 #  $\text{parity}(C_0) \oplus \text{parity}(C_1) = \text{parity}(C_4^{init})$  added to  $C_3$  ;
19 add_parity(s, 0, 3) ;
20 add_parity(s, 1, 3) ;
21 #  $\text{parity}(C_0) \oplus \text{parity}(C_2) \oplus \text{parity}(C_4) = \text{parity}(C_2^{init})$  added to  $C_1$ ;
22 add_parity(s, 0, 1) ;
23 add_parity(s, 2, 1) ;
24 add_parity(s, 4, 1) ;
```

Algorithm 2: add_parity(s , a , b)

Data: internal state s , column index a , column index b .

Result: internal state s where the parity of the column with index a has been added to the elements of the column with index b

```
1 for  $i$  in range(1, 5) do
2   | for  $j$  in range(5) do
3   |   |  $CNOT(5 * j + a, 5 * i + b)$ ;
4   | end
5 end
```

behaviour is dictated by the hardware topology. On a hardware with an all-to-all qubit connectivity, or with an emulator reproducing these conditions, the SWAP gate is replaced by a reindexation of the qubits by the transpiler³. On a hardware without all-to-all qubit connectivity, the transpiler replaces the SWAP gate by 3 CNOT gates.

The π function can be implemented without using any ancillary qubit due to the circular nature of the operation. The reverse π operation is straightforward as it only requires performing the swaps *mutatis mutandis* using $s[i][j] \leftarrow s[2i + 3j \bmod 5][i]$.

Algorithm 3: π algorithm for quantum hardware

```

1  $i \leftarrow 1$ ;
2  $j \leftarrow 0$ ;
3 for  $k$  in  $\text{range}(22)$  do
4   SWAP( $s[i][j]$ ,  $s[j][2i + 3j \bmod 5]$ );
5    $tmp \leftarrow j$ ;
6    $j \leftarrow i$ ;
7    $i \leftarrow 3(tmp - 3i) \bmod 5$ ;
8 end
9 #Last swap;
10 SWAP( $s[i][j]$ ,  $s[j][2i + 3j \bmod 5]$ );
```

2.3 χ function

The χ function is the only non-linear operation in Keccak, ensuring that the hash output is not a linear combination of the input and that the hash function is hard to inverse. In the case of Keccak-f(25) the formula is

$$s[i][j] \leftarrow s[i][j] \oplus (NOT(s[i + 1][j]) AND s[i + 2][j]) ,$$

which can be implemented on quantum hardware using two NOT gates and a Toffoli CCNOT gate.

We perform this operation line by line in the matrix representation of the internal state, using the in-place circuit from [LKL⁺24]. The reverse operation is performed by applying the same steps in reverse order.

Algorithm 4: χ algorithm for quantum hardware

```

1 for  $j$  in  $\text{range}(5)$  do
2    $s[0][j] \leftarrow s[0][j] \oplus (NOT(s[1][j]) AND s[2][j])$ ;
3    $s[2][j] \leftarrow s[2][j] \oplus (NOT(s[3][j]) AND s[4][j])$ ;
4    $s[4][j] \leftarrow s[4][j] \oplus (NOT(s[0][j]) AND s[1][j])$ ;
5    $s[1][j] \leftarrow s[1][j] \oplus (NOT(s[2][j]) AND s[3][j])$ ;
6    $s[0][j] \leftarrow s[0][j] \oplus (NOT(s[1][j]) AND s[2][j])$ ;
7    $s[3][j] \leftarrow s[3][j] \oplus (NOT(s[4][j]) AND s[0][j])$ ;
8    $s[0][j] \leftarrow s[0][j] \oplus (NOT(s[1][j]) AND s[2][j])$ ;
9 end
```

2.4 ι and ρ functions

The ρ function is a circular permutation along the lanes of the internal state (see [BDPA11]). However, in the case of Keccak-f(25), we operate on one 5×5 slice, hence the lanes only have one element. It follows that the ρ function for Keccak-f(25) is the identity.

The ι function in Keccak-f(25) is reduced to a bitflip of the bit at (0, 0) for rounds $\{1, 5, 6, 7, 8, 11\}$. This can be hardcoded by adding a CNOT gate to the circuit for the specific rounds concerned. The function is also its own inverse.

³The transpiler adapts the code to the topology of a specific backend, such as its instruction set and its qubit properties.

Algorithm 5: ι algorithm for quantum hardware

Data: Round number
1 # The rounds for which the round constant is odd is kept in the list;
2 **if** *round number* in $\{1, 5, 6, 7, 8, 11\}$ **then**
3 | $s[0][0] \leftarrow \text{NOT}(s[0][0]);$
4 **end**

3 Computational resources

3.1 Emulator

We report the computational resources needed to implement Keccak- $f(25)$ on the emulators using Qiskit AerSimulator 0.17. We express the gate count in terms of logical gates (CCNOT, CNOT, NOT) in Figure 3, and ISA gates (CZ, RZ, SX, X), which are the gates available on the IBM machines ibm-torino, ibm-marrakesh, and ibm-fez. The cost of each function in terms of logical gates is the same as for its reversed version. Note that the gate-count differences before and after the transpile are either due to the SWAPs being changed to gate-free qubit reindexations on the theoretical all-to-all qubit connectivity emulator, or to sequences of single-qubit gates being compressed into one gate (for example, the succession of two NOTs applied on one qubit is the identity). The depth of the circuit for one round or twelve rounds is also less than the sum of the depth of each elementary circuit θ, π, χ, ι taken independently, as some operations at the end of one circuit and at the beginning of the following one can be performed simultaneously when they do not use the same qubits.

Function	Gate count	Depth
	Before, (After) transpile	Before, (After) transpile
θ	450 CNOT (450 CNOT)	94 (94)
π	69 CNOT (None)	69 (0)
χ	35 CCNOT, 70 NOT (35 CCNOT, 60 NOT)	11 (10)
ι	1 NOT (1 NOT)	1 (1)
one round	35 CCNOT, 519 CNOT, 71 NOT	168 (104)
$\iota \circ \chi \circ \pi \circ \theta$	(35 CCNOT, 450 CNOT, 61 NOT)	
12 rounds	420 CCNOT, 6228 CNOT, 840 NOT (420 CCNOT, 5400 CNOT, 726 NOT)	1972 (1226)

Figure 3: Logical-gate count and depth in the basis (CCNOT, CNOT, NOT) of the Keccak operations θ, π, χ, ι , of one round $\iota \circ \chi \circ \pi \circ \theta$ and of twelve round of our 25-qubit Keccak- $f(25)$ implementation on the AerSimulator 0.17 emulator. Keccak operations and their reverse have the same logical-gate cost on the AerSimulator.

It appears from the logical-gate count that the most computationally expensive function in terms of gates is χ for its use of Toffoli gates CCNOT.

We compare these results with the gate count on a ISA-gates basis (CZ, RZ, SX, X) in Figure 4. The gate-basis (CZ, RZ, SX, X) gives a lower-level representation of the circuit, in which the gate count and the depth between the function and its reverse can slightly vary due to the possible gate combinations (typically for θ and χ). This difference remains under 10% for the depth. The ISA-gate count provides a more accurate depth of χ , unlike the logical-gate count, which hides it in the count of high-level CCNOTs.

3.2 Quantum hardware

The limited number of qubits required by our implementation of Keccak- $f(25)$ allows it to be run on quantum hardware. For our experiment, we used the IBM Quantum Platform under their free plan, using the machines ibm-torino, ibm-marrakesh, and ibm-fez, which have a Heron architecture. Their instruction set contains the gates SX, CZ, RZ and X. Quantum gate counts are trickier than with the emulator: unlike emulators for which the gate count is deterministic, the gate count on quantum hardware varies from one execution to another depending on the quantum machine used, on the topology of the machine, and on the qubits available at the time of the execution on the

Function	Gate count After transpile	Depth
θ	450 CZ, 115 RZ, 90 SX	119
reversed θ	450 CZ, 115 RZ, 90 SX	117
π	0	0
reversed π	0	0
χ	210 CZ, 465 RZ, 420 SX, 40 X	164
reversed χ	210 CZ, 480 RZ, 420 SX, 55 X	153
ι	1 NOT	1
reversed ι	1 NOT	1
one round $\iota \circ \chi \circ \pi \circ \theta$	660 CZ, 557 RZ, 510 SX, 36 X	280
reversed one round	660 CZ, 560 RZ, 498 SX, 52 X	267
12 rounds	7920 CZ, 6409 RZ, 6120 SX, 377 X	3206
reversed 12 rounds	7920 CZ, 6451 RZ, 5976 SX, 563 X	3094

Figure 4: ISA-gate count and depth in the basis (CZ, RZ, SX, X) of the Keccak operations θ , π , χ , ι , of one round $\iota \circ \chi \circ \pi \circ \theta$ and of twelve round of our 25-qubit Keccak- $f(25)$ implementation on the AerSimulator 0.17 emulator.

machine. Each machine has its own instruction set and a different way to map logical qubits onto its physical qubits. Besides, not all qubits are connected with one another. This is the reason why the execution of the code on a quantum hardware requires a transpiling step which maps the logical qubits and logical gates as efficiently as possible onto the available physical qubits and the machine’s instruction set.

Figure 5 reports an empirical gate count and depth to provide an indication of the number of ISA gates required to run Keccak- $f(25)$ on quantum hardware. However, variations should be expected. The comparison between the ISA-gate count of emulators and quantum hardware shows an increase in the number of gates and in the depth of the circuit. For one round, resp. twelve rounds of Keccak- $f(25)$, the depth is increased by a factor 13.4, resp. 15.5 between the emulator and the quantum hardware. This varies depending of the functions: while ι is not affected, the depth is increased by a factor 22 for θ . This highlights that not only the depth of the circuit but the nature of the operations should be taken into account to estimate the resources needed on a quantum hardware from an emulator-based ISA-gate count. Note that the SWAP gates have been replaced by 3 CNOT gates by the transpiler due to the mostly linear architecture of the Heron backend.

Function	Operations used	Depth	Qiskit Runtime (s)
θ	3405 CZ, 1760 RZ, 6053 SX	2695	3
reversed θ	3456 CZ, 1764 RZ, 6165 SX, 6 X	2655	3
π	69 CZ, 138 SX	139	2
reversed π	69 CZ, 138 SX	139	2
χ	429 CZ, 716 RZ, 788 SX, 32 X	412	2
reversed χ	402 CZ, 718 RZ, 730 SX, 27 X	253	2
ι	1 X	1	2
One round	4386 CZ, 2748 RZ, 7988 SX, 18 X	3738	3
One round reversed	4443 CZ, 2553 RZ, 8070 SX, 19 X	3405	3
Twelve round	55734 CZ, 30650 RZ, 101508 SX, 139 X	49753	12

Figure 5: Empirical gate-count, depth, and runtime of each function of our Keccak- $f(25)$ implementation using 25 qubits on a quantum hardware with heron architecture (ibm-fez, ibm-marrakesh, ibm-torino). The timings are reported using the Qiskit Runtime of the execution for 4096 shots.

Being able to run Keccak- $f(25)$ on a quantum hardware also allows us to report the correctness of each operation, and to track when the correct result is not readable from the quantum output repartition anymore. We analyse each function and subfunction independently and provide the percentage of correct outputs after one execution of 4094 shots on the quantum devices. The results are reported in Figure 6. From our experiment, it appears that circuits with a depth of up to 500 do provide the expected answer; the accumulation of error for functions with a greater

depth renders the result unreadable.

Function	ISA-gates depth	Shots with correct answers	Most probable answer is correct
θ	2695	0.0%	No
add parity for one column	88	65.9%	Yes
compute parity for one column	29	79.3%	Yes
π	139	24.5%	Yes
χ	412	2.1%	Yes
χ for one line	280	58.7%	Yes
ι	1	66.1%	Yes

Figure 6: Correctness of each operation and sub-functions of our Keccak- $f(25)$ implementation using 25 qubits on a quantum hardware with Heron architecture (ibm-fez, ibm-marrakesk and ibm-torino) for 4096 shots.

4 Grover’s algorithm on Keccak- $f(25)$

Having a quantum implementation for Keccak- $f(25)$, we build an oracle and use it within Grover’s algorithm.

4.1 Grover’s algorithm

Grover’s algorithm [Gro96] enables searching an unsorted database with a quadratic speedup in the number of queries compared to the best-known classical methods. Given a function f and an output b in the codomain of f , Grover’s algorithm can be used to retrieve a preimage a in the domain of f such that $f(a) = b$ with high probability. The algorithm consists of two main stages, the *oracle* and the *diffusion* operator, which are alternately applied a number of times proportional to $\sqrt{N}$, where N is the size of the search space (see Figure 7).

In the initialisation, the search space is mapped onto a uniform quantum superposition, where each input qubit has an equal probability of being 0, resp. 1. For a preimage of size m , this means that we need a qubit register of size m placed in uniform superposition. After the initialization procedure, the algorithm begins with the application of the oracle. The oracle is an abstract operator that marks the target state by modifying its relative phase. In practice, for the problem of interest, this is implemented by applying the hash transformation to the computational register, then performing a multi-controlled phase shift marking the state of interest, and finally restoring the original state of the register by applying the inverse of the hash transformation f (see Figure 8). The inverse hash transformation then restores the uniform structure of the superposition, apart from the phase change applied to the target state. In the second stage, the diffusion operator amplifies the probability amplitude of the marked state while restoring its phase relative to the rest of the superposition (see Figure 9). By repeatedly alternating the oracle and diffusion steps, the probability of measuring the correct solution can be made arbitrarily close to one. The number of iterations must be chosen carefully because the amplification process is cyclic: if the procedure is repeated beyond the point that maximises the probability amplitude, the success probability begins to decrease, reducing the overall efficiency of the algorithm. The number of iterations that

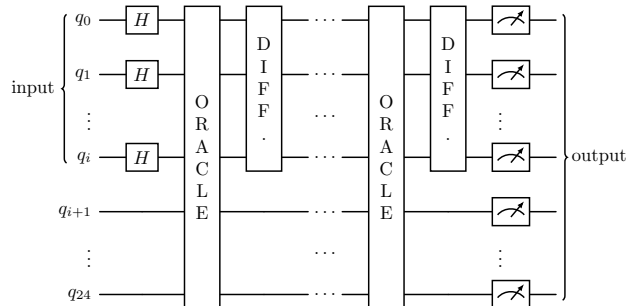


Figure 7: Grover’s algorithm iterations

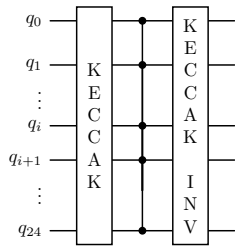


Figure 8: Grover’s oracle algorithm for Keccak-f(25)

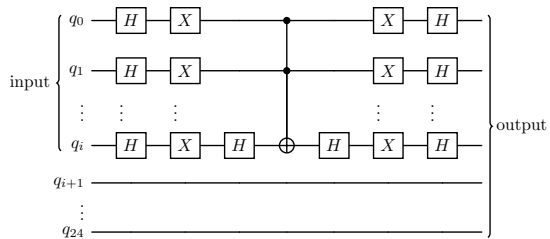


Figure 9: Grover’s diffusion algorithm for Keccak-f(25)

maximizes the probability of measuring the desired state is $\left\lfloor \frac{\pi}{4} \sqrt{\frac{N}{M}} \right\rfloor$, where N is the size of the search space and M is the number of acceptable inputs in the case of collisions (see [Gro96]). Regarding the marking of the state corresponding to the target, two options are generally considered: one uses an in-place phase shift of the target state, the other uses an ancillary qubit. Since we aim to minimise the number of qubits used, we chose to use the in-place phase shift marking using a multicontrolled Z gate.

4.2 Computational resources for Grover’s algorithm on Keccak-f(25)

We provide the logical and ISA gate count as well as the circuit depth needed to implement Grover’s algorithm on Keccak- $f(25)$ on an emulator using Qiskit AerSimulator 0.17. For our experiment, we use the following parameters for the sponge function (see Figure 1 and [BDPA11]): we choose the rate $r = 16$, the capacity $c = 25 - r = 9$, an input of size 8, and an output of size 16. We also take into account the padding applied to the input. Note that the size of the rate r and capacity c only affects on which qubits the padding is applied. The size of the input affects the number of qubits initially placed in superposition and hence both the number of gates used in the diffusion operator, and the optimal number of iteration to maximise the probability of measuring the correct answer, in this case $\lfloor \frac{\pi}{4} \sqrt{2^8} \rfloor = 12$. On the other hand, the size of the output affects the number of controlled phase shift gates in the oracle select step. The rest of the operations, that is, the Keccak and reversed Keccak operations, are unaffected by the rate or size of the input or output.

Our implementation aims at minimising the number of qubits used, such that it works both on the AerSimulator 0.17 and on quantum hardware. For the emulator, although a CPU (Intel Core i7 CPU with 16GB) can emulate the circuit, the use of GPUs is recommended to speed up the computation [Che25].

We report gate counts and depth of the resulting circuit on the AerSimulator 0.17, both with the logical-gate set CCNOT, CNOT, NOT, H, P in Figure 10 and with the ISA-gate set CZ, RZ, SX, X, H in Figure 11.

Function	Operations used	Depth
Oracle	840 CCX, 10950 CX, 1496 X, 100 H, 198 P	2616
Diffusion	50 CX, 18 X, 36 H, 53 P	122
One iteration	840 CCX, 11000 CX, 1514 X, 144 H, 251 P	2739
Optimal number of iterations (12)	10080 CCX, 132000 CX, 18146 X, 1640 H, 3012 P	32846

Figure 10: Logical-gate count and depth of Grover’s algorithm on Keccak-f(25) with our 25-qubit Grover’s algorithm on Keccak- f (25) implementation, on the AerSimulator 0.17 emulator, with rate $r = 16$, $c = 25 - r = 9$, an input of size 8 and an output of size 16.

We also run the circuit on the IBM Quantum Platform, using the machines `ibm-torino`, `ibm-marrakesh`, and `ibm-fez`, and report the ISA-gate count and depth of the circuit in Figure 12. We can observe from the results that the depth of the circuit is multiplied by a factor 14.9 (resp. 14.8) between the theoretical implementation using ISA gates on an emulator and the implementation on the quantum device for 12 iterations (resp. 1 iteration). This is due to the cost of adapting the circuit to the Heron architecture of the quantum machines, where qubits are mostly connected in lines. While the number of gates and the depth of the circuit makes the execution on a quantum

Function	Operations used	Depth
Oracle	16002 CZ, 13159 RZ, 12508 SX, 935 X	6631
Diffusion	50 CZ, 103 RZ, 122 SX, 16 X	213
One iteration	16052 CZ, 13262 RZ, 12630 SX, 951 X	6844
Optimal number of iterations (12)	192624 CZ, 158968 RZ, 151472 SX, 11390 X,	82084

Figure 11: ISA-gate count and depth of Grover’s algorithm on Keccak-f(25) with our 25-qubit Grover’s algorithm on Keccak- $f(25)$ implementation, on the AerSimulator 0.17 emulator, with rate $r = 16$, $c = 25 - r = 9$, an input of size 8, and an output of size 16.

computer too noisy to be able to read a correct result, we believe it is still useful to give an insight into the difference between the theoretical and ISA gate counts, the importance of the quantum hardware underlying architecture, and an idea of the order of magnitude of gates needed to run Grover’s algorithm on a hash function, even as small as Keccak-f(25).

Function	ISA-gate count	Depth	Qiskit runtime (s)
One iteration	116498 CZ, 61346 RZ, 212866 SX, 233 X	101402	15
Optimal number of iterations (12)	1381935 CZ, 733826 RZ, 2521555 SX, 2793 X	1223814	150

Figure 12: Empirical gate-count, depth and Qiskit runtime of each function of our Keccak- $f(25)$ implementation on a quantum hardware (ibm-fez, ibm-torino). The timing are reported using the Qiskit Runtime of the execution for 4096 shots.

5 Specifity of quantum password cracking with Grover’s algorithm

5.1 Search space reduction techniques

By studying leaked password databases, researchers have identified common patterns in user-generated passwords with regard to length, structure, and content [ASIM21]. Generally, user-generated passwords have a limited length, rarely exceeding 15 characters. Additionally, they tend to have a typical structure, such as *string-digit*, where a sequence of characters is followed by digits. Furthermore, passwords often contain meaningful sequences of characters, such as personal names. By combining these features, it is possible to create dictionaries of highly probable passwords. Theoretically, the asymptotic quadratic advantage of Grover over classical search methods also holds with a password guessing approach using a reduce set of possibilities, as shown in [DGM⁺21].

Dictionary attacks In practice, a quantum dictionary attack would require to initialise the qubits in a quantum state corresponding to the dictionary of probable password before applying Grover’s algorithm: for n qubits in superposition, that is 2^n possibilities, and a dictionary of D words of size less than n bits, this means creating a quantum state where the D dictionary words have a probability $\frac{1}{D}$ of being measured, while the $(2^n - D)$ words not in the dictionary are measured with probability 0. The authors from [MSRM20] provide an algorithm to create such a distribution, which requires $(2^n - 2)$ CNOT gates and $(2^n - 1)$ single-qubit gates in the worst case, but the cost varies with the dictionary chosen. This initialisation phase would have to be reversed before applying Grover’s diffusion operator, and the qubits reinitialised again after the diffusion. Writing $\mathcal{C}$ the cost of the initialisation, and assuming that the reversed initialisation has the same cost, the cost of each Grover’s iteration with a dictionary attack is therefore increased by $2\mathcal{C}$ compared to the undifferentiated case. However, the search space is reduced when using a dictionary, meaning that the optimal number of iterations is reduced from $\frac{\pi}{4}\sqrt{2^n}$ to only $\frac{\pi}{4}\sqrt{D}$. All in all, with the notations above, if the cost of one Grover’s iteration in the undifferentiated case is written $\mathcal{G}$, the cost of a quantum dictionary attack would be $\lfloor \frac{\pi}{4}\sqrt{D} \rfloor (\mathcal{G} + 2\mathcal{C})$, whereas the

cost of the undifferentiated attack would be $\lfloor \frac{\pi}{4} \sqrt{2^n} \rfloor (\mathcal{G})$. For the quantum dictionary attack to be more efficient than the undifferentiated approach, the cost of the initialisation $\mathcal{C}$ and the number D of elements in the dictionary must be such that

$$\mathcal{C} < \frac{\mathcal{G}}{2} \sqrt{\frac{2^n}{D}} - 1 .$$

Mask attacks Instead of relying on a dictionary of probable passwords, another approach is to target specific patterns of the password, for example the position of upper-case, lower-case or digit characters. By making some pattern assumptions, we reduce the number of qubits placed in superposition in the initialization and therefore reduce the size of the search space without having to perform additional initialisation steps. For example, assuming that the first letter is an uppercase letter, or that the last one is a number, might fix some of the qubits to 0 or to 1 depending on the encoding used. In ASCII, upper-case letters start with 010, lower-case letters with 011, and numbers start with 0011. By reducing the number of possible pre-images and hence the search-space size, this approach reduces the number of iterations needed for Grover’s algorithm. It is the approach we adopt in this work.

5.2 Mask attack experiment

Keccak is rarely used to store passwords in practice, given that specific password-storing functions exist, and should be used instead. Nonetheless, our implementation of Keccak-f(25) allows us to study in practice how search space reductions can be used within Grover’s algorithm for quantum password guessing.

If SHA-3 was used for password storage, the size of the password is likely to be smaller than the rate of the sponge function, and only one iteration of the sponge function would be needed to absorb it. For our choice of parameters in the experiment, we reproduce at a smaller scale the ratios between the size of the password, the size of the hash output and the choice of rate used in SHA-3. For our experiment, we choose parameters that are a small scale reproduction of the situation were Grover’s algorithm would be used to recover the preimage of a SHA-3 hash of a password: we choose a rate $r = 16$ about twice the size of the capacity $c = 25 - r = 9$, an input of size 8 smaller than the rate, and an output size 16 larger than the input size. We also take into account the padding applied to the input.

The input size is 8 bits, which is one ASCII character. Given that the input size is smaller than the output size, we assume that collisions are unlikely. The number of iterations that maximises the probability of measuring the correct result when considering the space of all 2^8 possibilities is therefore $\lfloor \frac{\pi}{4} \sqrt{2^8} \rfloor = 12$. However, if we assume that the input character is in lower-case, resp. upper-case, then three bits of the input are fixed, and the optimal number of Grover’s algorithm iterations is now $\lfloor \frac{\pi}{4} \sqrt{2^5} \rfloor = 4$. If we assume instead that the input character is a digit, the optimal number of iterations is reduced to 3.

We compare the logical-gate count and depth with and without an assumption about the pattern of the password in Figure 13. This approach can be extended to several characters for larger inputs, depending on the pattern of the password chosen.

Assumption	Iterations	Logical-gate count	Depth	Ratio
None	12	10080 CCX, 132000 CX, 18146 X, 1640 H, 3012 P	32846	1
Upper-case	4	3360 CCX, 43904 CX, 6019 X, 477 H, 908 P	10710	0.33
Lower-case	4	3360 CCX, 43904 CX, 6019 X, 477 H, 908 P	10710	0.33
Digit	3	2520 CCX, 32892 CX, 4504 X, 364 H, 675 P	7976	0.24

Figure 13: Logical-gate count and depth of Grover’s algorithm on Keccak-f(25) with our 25-qubit Grover’s algorithm on Keccak-f(25) implementation, on the AerSimulator 0.17 emulator, with rate $r = 16$, $c = 25 - r = 9$, an input of size 8 and an output of size 16, with and without assumption on the pattern of the input.

6 Conclusion

In this work, we build a 25-qubit in-place quantum implementation of Keccak-f(25), a toy version of SHA-3, which can run both on emulators and quantum hardware, with the source code publicly

available. In particular, we present, for the first time to the best of our knowledge, an in-place implementation of the Keccak-f(25) θ function.

We provide logical-gate and ISA-gate counts on emulators as well as ISA-gate counts on quantum hardware, both for Keccak-f(25) and Grover’s algorithm on Keccak-f(25). These counts highlight the dominant cost of the χ function, closely followed by θ in in-place implementations, the need to rely on the ISA-gate count to provide an accurate idea of the depth of the circuit, as well as the depth increase due to the map of logical qubits onto physical ones, depending on the qubits’ topology. We also report the correctness levels of the operations on the quantum hardware used (ibm-torino, ibm-marrakesh and ibm-fez, all with Heron architecture), showing that for circuits with a (ISA) depth less than 500, the most probable output over 4096 shots was indeed the correct answer.

Finally, we use our implementation to highlight the specificity of quantum password cracking, namely the use of pattern identification in passwords, or mask-attacks, to reduce the search space, and therefore the number of iterations of Grover’s algorithm needed to measure the correct result in the most probable output.

While the architecture and the noise-correction of quantum computers will evolve, this work provides a first benchmark of the actual capabilities of the machines tested, as well as an insight into the requirements to run Grover’s algorithm on a scaled-down version of SHA-3.

7 Acknowledgement

The authors would like to thank Iwen Coisel for his useful feedback, and anonymous reviewers for their suggestions.

References

- [AMG⁺16] Matthew Amy, Olivia Di Matteo, Vlad Gheorghiu, Michele Mosca, Alex Parent, and John M. Schanck. Estimating the cost of generic quantum pre-image attacks on SHA-2 and SHA-3. In *Selected Areas in Cryptography - SAC 2016*, volume 10532 of *Lecture Notes in Computer Science*. Springer, 2016.
- [ASIM21] Kanta A, Coray S, Coisel I, and Scanlon M. How viable is password cracking in digital forensic investigation? Analyzing the guessability of over 3.9 billion real-world accounts. *Forensic Science International - Digital Investigation*, 2021.
- [BDK15] Alex Biryukov, Daniel Dinu, and Dmitry Khovratovich. Argon2: the memory-hard function for password hashing and other applications. *Password Hashing Competition*, 2015.
- [BDPA11] Guido Bertoni, Joan Daemen, Michaël Peeters, and Gilles Van Assche. The Keccak reference, 2011.
- [Che25] Mathilde Chenu. Quantum emulators: CPU, single GPU and multiple GPUs performance comparison. *Procedia Computer Science*, 2025. Proceedings of the Third EuroHPC user day.
- [CPB⁺] Shu-Jen Chang, Ray Perlner, William Burr, Meltem Sönmez Turan, John Kelsey, Souradyuti Paul, and Lawrence Bassham. Third-Round Report of the SHA-3 Cryptographic Hash Algorithm Competition, NISTIR 7896. National Institute of Standards and Technology.
- [CSYS25] Sangmin Cha, Gyeongju Song, Seyoung Yoon, and Hwajeong Seo. Quantum implementation of MD5. *IACR Cryptol. ePrint Arch.*, page 1572, 2025.
- [DBK24] Prodipto Das, Sumit Biswas, and Sandip Kanoo. Quantum implementation of SHA1 and MD5 and comparison with classical algorithms. *Quantum Inf. Process.*, 2024.
- [DC24] Sarah D. and Peter C. On the practical cost of Grover for AES key recovery. UK National Cyber Security Centre, 2024.
- [DGM⁺21] Markus Dürmuth, Maximilian Golla, Philipp Markert, Alexander May, and Lars Schlieper. Towards Quantum Large-Scale Password Guessing on Real-World Distributions. In Mauro Conti, Marc Stevens, and Stephan Krenn, editors, *Cryptology and Network Security - CANS 2021*, Lecture Notes in Computer Science. Springer, 2021.

- [GE25] Ramin Rezvani Gilkolaei and Reza Ebrahimi. Quantum Resource Analysis of Low-Round Keccak/SHA-3 Preimage Attack: From Classical $2^{57.8}$ to Quantum $2^{28.9}$ using Qiskit Modeling. *IACR Cryptol. ePrint Arch.*, 2025.
- [GLRS16] Markus Grassl, Brandon Langenberg, Martin Roetteler, and Rainer Steinwandt. Applying Grover’s Algorithm to AES: Quantum Resource Estimates. In *Post-Quantum Cryptography - PQCrypto 2016*, Lecture Notes in Computer Science. Springer, 2016.
- [GM25] Vlad Gheorghiu and Michele Mosca. Quantum resource estimation for large scale quantum algorithms. *Future Gener. Comput. Syst.*, 2025.
- [Gro96] Lov K. Grover. A Fast Quantum Mechanical Algorithm for Database Search. In *Proceedings of the Twenty-Eighth Annual ACM Symposium on the Theory of Computing 1996*. ACM, 1996.
- [JLO⁺25] Kyungbae Jang, Sejin Lim, Yujin Oh, Hyunjun Kim, Anubhab Baksi, Sumanta Chakraborty, and Hwajeong Seo. Quantum implementation and analysis of SHA-2 and SHA-3. *IEEE Trans. Emerg. Top. Comput.*, 2025.
- [JNRV20] Samuel Jaques, Michael Naehrig, Martin Roetteler, and Fernando Virdia. Implementing grover oracles for quantum key search on AES and LowMC. In *Advances in Cryptology - EUROCRYPT 2020*, Lecture Notes in Computer Science. Springer, 2020.
- [KHJ18] Panjin Kim, Daewan Han, and Kyung Chul Jeong. Time-space complexity of quantum search algorithms in symmetric cryptanalysis: applying to AES and SHA-2. *Quantum Information Processing*, 2018.
- [LJS⁺22] Wai-Kong Lee, Kyungbae Jang, Gyeongju Song, Hyunji Kim, Seong Oun Hwang, and Hwajeong Seo. Efficient Implementation of Lightweight Hash Functions on GPU and Quantum Computers for IoT Applications. *IEEE Access*, 2022.
- [LKL⁺24] Jongheon Lee, Yousung Kang, You-Seok Lee, Boheung Chung, and Doocho Choi. Toffoli-depth reduction method preserving in-place quantum circuits and its application to SHA3-256. *Quantum Inf. Process.*, 2024.
- [MSRM20] Fereshte Mozafari, Mathias Soeken, Heinz Riemer, and Giovanni De Micheli. Automatic Uniform Quantum State Preparation Using Decision Diagrams. In *50th IEEE International Symposium on Multiple-Valued Logic, ISMVL*, pages 170–175. IEEE, 2020.
- [NIS] NIST National Institute of Standards and Technology. Federal Information Processing Standard Publication Secure Hash Standard (SHS) - FIPS PUB 180-4. National Institute of Standards and Technology.
- [PMH08] Ketan N. Patel, Igor L. Markov, and John P. Hayes. Optimal synthesis of linear reversible circuits. *Quantum Inf. Comput.*, 2008.
- [Pre22] Richard H. Preston. Applying Grover’s Algorithm to Hash Functions: A Software Perspective. *IEEE Transactions on Quantum Engineering*, 2022.
- [RBL⁺21] Sergi Ramos-Calderer, Emanuele Bellini, José I. Latorre, Marc Manzano, and Víctor Mateu. Quantum search for scaled hash function preimages. *Quantum Inf. Process.*, 2021.
- [Riv92] Ronald Rivest. The MD5 Message-Digest Algorithm, 1992.
- [SJS23] Gyeongju Song, Kyungbae Jang, and Hwajeong Seo. Improved Low-depth SHA3 Quantum Circuit for Fault-tolerant Quantum Computers. *Appl. Sci.*, 2023.
- [SP25] Yeonsong Suh and Youngho Park. Resource-Efficient Reversible Quantum Circuit for Theta Function in SHA-3. *2025 Sixteenth International Conference on Ubiquitous and Future Networks (ICUFN)*, 2025.
- [WWJ⁺23] Dennis Willsch, Madita Willsch, Fengping Jin, Hans De Raedt, and Kristel Michielsen. Large-Scale Simulation of Shor’s Quantum Factoring Algorithm. *Mathematical Perspectives on Quantum Computing and Communication*, 2023.
- [YSJ⁺25] Seyoung Yoon, Gyeongju Song, Kyungbae Jang, Sangmin Cha, and Hwajeong Seo. Quantum implementation of SHA-1. *IACR Cryptol. ePrint Arch.*, 2025.